

武汉大学

WUHAN UNIVERSITY



F7LY-OS 设计文档

参赛队名	F7LY
队伍成员	曹子宸、郑喆宇、官恺祺
指导老师	蔡朝晖

目 录

第一章 概 述	1
1.1 F7LY 内核的介绍	1
1.2 整体架构	2
第二章 内核详细介绍	4
2.1 机器启动	4
2.1.1 RISC-V 启动流程	4
2.1.2 LoongArch 启动流程	6
2.1.3 双架构支持特色	7
2.2 中断管理器	7
2.3 内存管理	9
2.3.1 地址空间	9
2.3.2 物理内存管理	11
2.4 进程管理	18
2.4.1 进程控制块 (PCB)	18
2.4.2 进程调度	20
2.4.3 线程的支持	22
2.4.4 进程操作与管理	23
2.5 文件系统架构	25
2.5.1 虚拟文件系统	25
2.5.2 VFS 核心元数据结构剖析	26
2.5.3 支持的文件系统	32
2.6 进程间通信	33
2.6.1 信号机制	33
2.6.2 信号处理函数	34
2.6.3 Futex	35
第三章 总结与展望	38
3.1 工作总结	38
3.2 经验总结	38
3.3 未来计划	39
附录 A 一些重要的类的具体字段解析	40

第一章 概述

1.1 F7LY 内核的介绍

F7LY-OS 是 C++ 编写的支持 RISC-V 与 LoongArch 双架构的宏内核模块化操作系统，由来自武汉大学的三名队员使用 C++ 从 0 参考 xv6 并改编、增加功能构建而成。使用分体的硬件抽象层实现架构无关的内核核心功能，进行多阶段启动，提供统一的进程管理，细粒度的内存资源动态管理，拥有统一接口的虚拟文件系统，可使用 C++ 的多态特性支持 fat32 与 ext4，以及基础的信号机制。开发过程的全部 commit 记录请参见原仓库地址 [KidsZZ/F7LY-OS]。^①

文件系统和 LoongArch 架构的内存管理部分参考了 2024 年武汉大学队伍 [XN6]^② 的实现；异常处理逻辑与内核地址布局设计借鉴了 MIT S081 的 [xv6-2021]^③ 项目。在开发过程中，F7LY 未使用标准 C++ 库，而是引入并采用了第三方库 EASTL 以获得必要的数据结构支持。堆内存分配方面，F7LY 参考并集成了 Liballoc 相关代码，以实现高效的堆空间管理。

F7LY 使用模块化、面向对象的方式组织内核代码，将宏内核各个部分独立分成不同的模块。通过明确模块之间的接口，减少模块间的相互依赖和干扰，从而使内核代码复用成为可能，目标是提供功能与性能兼备的模块以及可读性强、可迁移性强的代码。

F7LY 致力于实现高质量的代码，使内核充分发挥 C++ 编译型语言的优点，接近硬件层的代码书写与直接操作内存的特性可更直观地操作内核中的资源管理。功能方面，实现了完整的 C++ ABI 支持，包括异常处理机制，支持标准 POSIX 系统调用；性能方面，通过懒分配和写时复制优化 execve 执行速度，使用页缓存加速文件读写，减少 I/O 访问次数。

F7LY 严格遵循 RISC-V 与 LoongArch 的硬件标准与中断处理规范，使用统一的 trap 管理框架支持 RISC-V PLIC 和 LoongArch 的 PLIC/ExtIOI 中断控制器，来正确处理系统调用和各类例外与中断。系统调用严格按照 System Calls Manual 手册进行开发，确保实现的所有系统调用符合 Linux raw syscall 规范，为运行在 F7LY 的用户态程序提供了完备可靠的系统调用支持。

① <https://github.com/KidsZZ/F7LY-OS>

② <https://gitlab.eduxiji.net/T202410486992576/OSKernel2024-2k1000la-xv6>

③ <https://github.com/mit-pdos/xv6-public>

1.2 整体架构

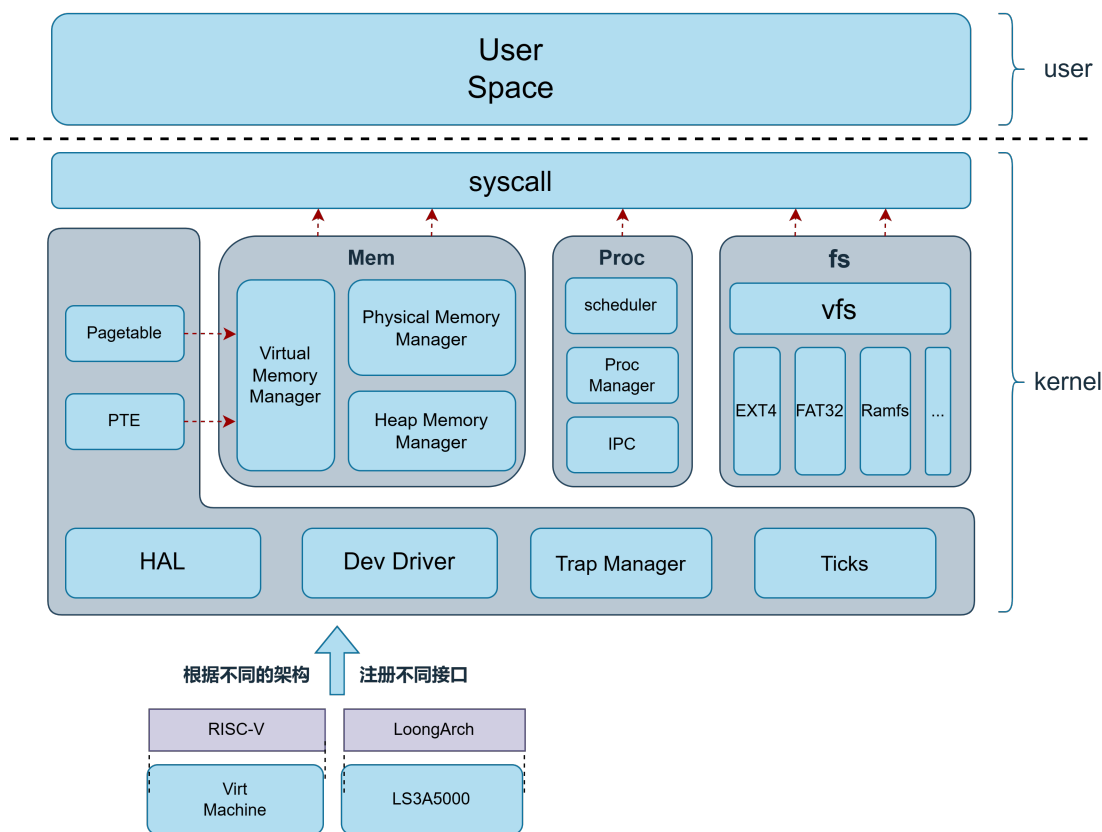


图 1-1 整体架构

kernel 目录中对代码进行了模块化分区，支持双架构设计。

- **boot 系统启动模块** - 负责内核的启动流程，包含 RISC-V 和 LoongArch 两种架构的启动代码，实现从 bootloader 到内核 main 函数的跳转，初始化基础硬件环境。
- **devs 设备管理模块** - 实现统一的设备抽象层，包含字符设备、块设备、流设备等抽象类，提供设备管理器进行设备注册和管理，支持 UART、控制台、磁盘等硬件设备。
- **fs 文件系统模块** - 实现了 VFS（虚拟文件系统）层，支持多种文件系统包括 ext4、FAT、ramfs 等，提供统一的文件操作接口，包含缓冲区管理、目录项缓存、inode 管理等核心组件。
- **hal 硬件抽象层** - 提供跨架构的硬件抽象，封装 CPU 相关操作和上下文切换，为上层模块提供统一的硬件接口，支持 RISC-V 和 LoongArch 双架构。
- **libs 内核库模块** - 提供内核所需的基础库函数，包括字符串操作、打印输出、内存分配器、模板算法、C++ 运行时支持等，为内核其他模块提供基础服务。

- **mem** 内存管理模块 - 实现完整的内存管理体系，包含物理内存管理器（支持伙伴系统算法）、虚拟内存管理器、堆内存管理器、slab 分配器等，提供高效的内存分配和回收机制。
- **proc** 进程管理模块 - 实现进程创建、调度、同步等核心功能，包含进程管理器、调度器、信号处理、管道通信、锁机制等，支持多进程并发执行。
- **sys** 系统调用模块 - 实现系统调用处理机制，提供用户态程序与内核的交互接口，包含系统调用分发、参数传递、权限检查等功能。
- **trap** 中断与异常处理模块 - 处理硬件中断、异常和系统调用，实现中断向量表设置、异常处理流程、时钟中断管理等，支持双架构的不同中断机制。
- **tm** 时间管理模块 - 提供时间和定时器相关服务，实现定时器管理器、时间接口等，为进程调度和系统调用提供时间基础服务。
- **net** 网络模块 - 网络协议栈的基础框架（预留接口）。

F7LY-OS

```

├─ build                      //工程构建文件
├─ kernel                    //内核文件
│   ├── boot                 //内核入口以及主函数
│   ├── devs                 //设备驱动
│   ├── fs                   //文件系统
│   │   ├── dev              //文件相关设备
│   │   ├── ext4             //ext4文件操作
│   │   ├── fat              //fat文件操作
│   │   ├── jbd2             //日志管理
│   │   ├── ramfs            // ram文件系统
│   │   └─ vfs               //虚拟文件系统
│   └─ file                  //用于继承的统一文件操作接口
├─ hal                      //与平台相关的寄存器定义
├─ libs                      //自定义的库操作支持
├─ link                      //链接脚本
├─ mem                       //内存管理
├─ net                       //网络相关
├─ proc                      //进程调度与管理
├─ sys                       //系统调用
├─ tm                        //时钟
├─ trap                      //中断处理
├─ thirdparty                //使用的第三方库
│   └─ EASTL                 //EASTL库
├─ user                      //用户程序
└─ Makefile

```

第二章 内核详细介绍

2.1 机器启动

F7LY OS 支持双架构启动，机器启动的源文件分别在 `kernel/boot/riscv/` 和 `kernel/boot/loongarch/` 目录下。

启动流程遵循经典的多阶段启动模式：`Bootloader -> entry.S -> start.cc -> main.cc`

整个 `kernel` 可执行文件的入口点在各自架构的 `entry.S` 中的 `_entry` 标签，通过链接脚本指定。

RISC-V 架构：基于 SBI(Supervisor Binary Interface) 规范，Bootloader 工作在 M-mode，内核代码从 `entry.S` 开始运行在 S-mode。

LoongArch 架构：采用 LoongArch 原生启动方式，通过 ACPI 等标准接口进行硬件初始化。

2.1.1 RISC-V 启动流程

2.1.1.1 entry.S 阶段

- 负责设置操作系统的栈指针 `sp` 到合适位置，为后续 C++ 代码执行做准备。
- 每个 CPU 核心分配 4KB 独立栈空间，通过 `hartid` 进行区分。
- 完成栈设置后跳转到 `start()` 函数。

```
_entry:
    la sp, stack0           # Load base addr of stack
    li t0, 1024*4           # 4KB space per stack
    mv t1, a0               # gain hartid
    addi t1, t1, 1
    mul t0, t0, t1           # cal current offset of stack
    add sp, sp, t0           # set stack pointer
    call start              # jump to start func
```

2.1.1.2 start.cc 阶段

关键寄存器处理：

- `a0`：存储硬件线程编号 `hartid`。
- `a1`：设备树地址信息 `dtb_entry`。
- `sp`：已在 `entry.S` 中设置完成。

主要工作：

- 关闭分页机制，使用物理地址访问
- 设置临时 trap 处理函数为死循环
- 将 hartid 保存到 tp 寄存器供后续使用
- 调用 main() 函数进行系统初始化

```
void start(uint64 hartid, uint64 dtb_entry)
{
    riscv::csr::_write_csr_(riscv::csr::satp, 0);
    riscv::csr::_write_csr_(riscv::csr::stvec,
        ↪ (uint64)trap_loop);
    riscv::w_tp(hartid);    // Save hartid
    main();
}
```

2.1.1.3 主函数初始化

系统初始化分为三个主要阶段：

1. 基础服务初始化

```
k_printer.init();
trap_mgr.init();
plic_mgr.init();
```

2. 内存管理初始化

```
mem::k_pmm.init();
mem::k_vmm.init();
mem::k_hmm.init();
```

3. 进程和设备管理

```
proc::k_pm.init();
dev::k_devm.register_stdin();
riscv::qemu::disk_driver.init();
syscall::k_syscall_handler.init();
```

4. 启动用户进程和调度器

```
proc::k_pm.user_init();
proc::k_scheduler.start_schedule();
```

2.1.2 LoongArch 启动流程

loongarch 部分将 `start.cc` 阶段合并到 `entry.S` 中，在其中完成设置设备操作空间、设置指令数据访问空间，CSR 寄存器的初始化，以及栈空间的设定，并开启 fpu 浮点数指令。

entry 只对每个 CPU 设定自己的栈空间，然后跳转到 main 函数

```
li.d      $t0, 0x8000000000000001
csrwr     $t0, LOONGARCH_CSR_DMWIN0 # 设置设备操作空间
li.d      $t0, (0x9000000000000001 | 1 << 4)
csrwr     $t0, LOONGARCH_CSR_DMWIN1 # 设置指令数据访问空间
csrwr     $t0, LOONGARCH_CSR_TLBRENTY

li.w      $t0, 0xb0                # PLV=0, IE=0, PG=1
csrwr     $t0, LOONGARCH_CSR_CRMD
li.w      $t0, 0x00                # PPLV=0, PIE=0, PWE=0
csrwr     $t0, LOONGARCH_CSR_PRMD
li.w      $t0, 0x01                # FPE=1, SXE=0, ASXE=0, BTE=0
csrwr     $t0, LOONGARCH_CSR_EUEN
```

其余大部分启动的基本流程与 riscv 类似，在中断与虚拟化方面 LoongArch 架构采用不同的启动方式：

- 使用 ACPI 控制器进行硬件发现和初始化。
- 通过 APIC 和 ExtIOI 进行中断管理。
- 支持 LoongArch 原生的虚拟化和安全特性。

```
void main()
{
    k_printer.init();
    dev::acpi::k_acpi_controller.init();
    apic_init();
    extioi_init();
    // ... 后续与 RISC-V 类似的初始化流程
```



```
}
```

2.1.3 双架构支持特色

- **统一的抽象层**：通过 HAL(硬件抽象层) 实现跨架构代码复用。
- **架构特定优化**：针对不同架构的特性进行专门优化。
- **模块化设计**：核心模块与架构相关代码分离，便于维护和扩展。
- **现代化实现**：采用 C++ 面向对象设计，提供类型安全和更好的代码组织。

2.1.3.1 启动完成标志

系统启动完成后会显示欢迎信息并启动调度器：

```
=== SYSTEM BOOT COMPLETE ===
Kernel space successfully initialized
```

此时内核已完成所有初始化工作，进入正常的多进程调度运行状态。

2.2 中断管理器

从用户态到内核态的切换需要中断和异常的频繁处理，此处 F7LY 在迁移 xv6 代码的过程中选择了对象化中断管理器类 `TrapManager`，并实例化全局对象 `trap_mgr`。

```
class trap_manager
{
public:
    void init();
    void inithart();
    int devintr();          // 处理外部中断和软件中断
    void usertrap();
    void usertrapret();
    void machine_trap();
    void kerneltrap();
private:
    void timertick();
    SpinLock tickslock;
    uint ticks;
    uint timeslice;
```

```
};
```

F7LY 在其底层根据 RISC-V 和 loongarch 的不同硬件设备使用而封装了不同的中断处理器，在不同文件夹下实现了 PLIC、EXTIOI、APIC 的驱动并应用在中断处理之中。

- RISC-V 在处理中设定了中断处理的入口 (`uservec`) 以及返回的入口 (`user-trapret`), 具体逻辑参照了 xv6 的中断处理, 但封装在对象之中, 并使用包装函数 (`wrap_` 前缀的全局函数) 设置中断入口。
- Loongarch 针对架构的特殊性设置了 TLB 重填的处理程序入口 (`tlbrefill.S`), 以及机器异常的处理程序 (`merror.S`), 使用类似的逻辑进行接口统一化, 使得上层程序可以统一调用。

如果用户程序发出系统调用, 或者做了一些非法的事情, 或者设备中断, 那么在用户空间中执行时就可能会产生陷阱。来自用户空间的陷阱的高级路径是 `uservec`, 然后是 `usertrap`; 返回时, 先是 `usertrapret`, 然后是 `userret`。

首先通过 `crmd` 寄存器确认陷阱确实来自用户模式, 然后将中断入口点 (`eentry`) 设为 `kernelvec`, 并获取当前进程信息并保存用户程序计数器。

函数通过检查异常代码 (`CSR_ESTAT_ECODE`) 来区分不同类型的陷阱:

- **系统调用 (0xb):**
 - 检查进程是否被标记为结束。
 - 调整返回地址, 使其指向 `ecall` 指令之后的指令。
 - 启用中断, 然后调用系统调用处理器。
- **内存错误 (0x1 或 0x2):**
 - 调用 `mmap_handler` 来处理缺页异常。
 - 如果处理失败, 则标记进程为已终止。
- **设备中断:**
 - 调用 `devintr()` 处理设备中断。
- **未知陷阱:**
 - 打印错误信息并标记进程为已终止。

如果是时钟中断 (`which_dev == 2`), 则增加时间片计数, 当达到阈值 (10 个时间片) 时调用 `yield`。最后调用 `usertrapret()` 函数返回用户态, 该函数负责恢复用户态执行环境。

2.3 内存管理

2.3.1 地址空间

2.3.1.1 内核链接地址

F7LY 在支持多架构内核时，并未采用“一体适配”的简化策略，而是根据架构自身特性量身定制内核布局，以保障启动路径的最小依赖性和最大清晰性。

对于 RISC-V，qemu 的入口地址位于 `0x8020_0000`。在此处，F7LY 采用 OPENSBI 自动识别内核的 ELF 文件入口地址（LMA），低地址采用直接映射的方式映射所有 MMIO 设备以及内核代码。因为在进入到 `entry.S` 时仍然未开启分页，所以映射时低处地址采用直接映射便于访问。

与 RISC-V 使用 QEMU 默认跳转地址（`0x80200000`）不同，LoongArch 架构的启动顺序允许我们将内核加载地址与运行地址（ $VMA = LMA$ ）分离，从而更灵活地构造分页映射策略。F7LY 的 LoongArch 端应用 loongarch 的直接映射窗口，在链接脚本处使用 `0x9000000008000000` 加载内核镜像。

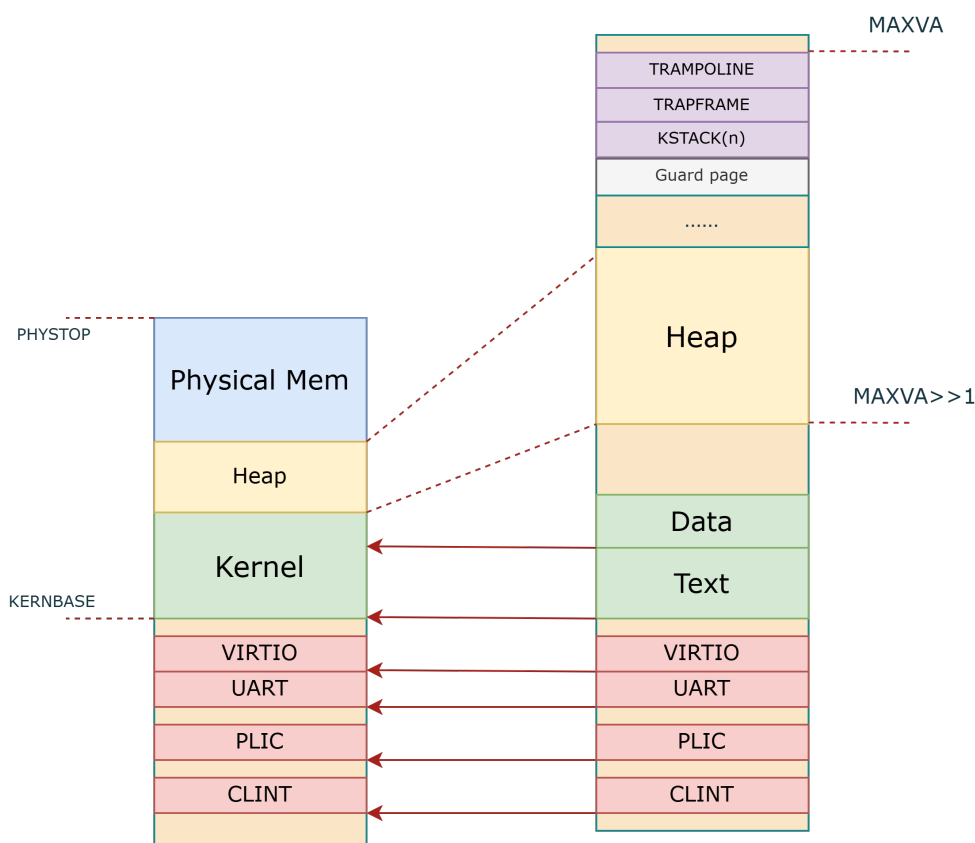


图 2-1 内核地址空间布局

2.3.1.2 地址空间布局

F7LY 的地址空间的设计参考了 xv6 的物理地址布局，内核态页表保存在全部内核地址空间 Kernel 中，其记录了内存的空间布局。用户页表在用户空间中记录用户态地址。

对于内核地址空间，为方便管理物理地址，从 KERNBASE 开始映射内核映像；堆内存从地址 `0x84000000` 开始，剩余的空闲物理地址交由物理内存管理器管理，用于后续的动态分配。

对于用户地址空间，为便于堆空间的增长和用户程序的动态访问，将用户地址的代码从 `Proc_base` 进行加载。每当用户对内存进行申请时，采用对指针的移动进行扩张。由于 loongarch 的加载地址与 riscv 不一致，我们在 `execve` 对程序进行加载时显式设置程序的空间开始地址（`PROC_BASE`），并从此处开始设置指针，装载完成后的指针位置既标志了进程此时的大小，又可作为堆顶指针开始动态内存分配。

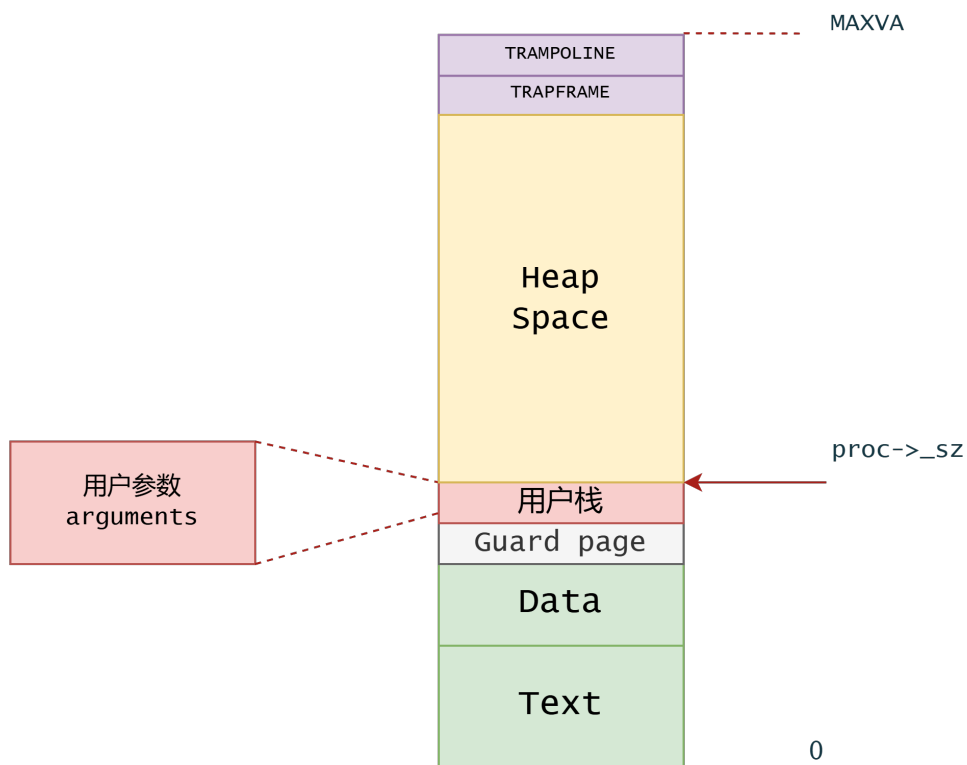


图 2-2 用户地址空间布局

2.3.2 物理内存管理

2.3.2.1 内核动态内存分配器

F7LY 使用粗细粒度混合分配管理内核所需的物理内存，统一使用伙伴内存分配器进行粗粒度分配。伙伴分配器用于管理一片连续内存，从一个地址开始初始化，并设置最多支持的分配粒度便可以指定伙伴分配器管理的一片连续内存。

```
#define PAGE_ORDER 10
#define PGNUM (1 << 15)
#define BSSIZE 10 //最多支持 2^10 = 1024 页的分配粒度

class BuddySystem {
public:
    int Alloc(int size);           //按照大小分配
    void Free(int offset);
    void* alloc_pages(int count); //按照页数分配
    void free_pages(void* ptr);
private:
    uint8* tree;                  //建立对一片内存管理的树结构
    uint8* base_ptr;              //管理内存的起始地址
}
```

伙伴分配器 (Buddy Allocator) 通过分配和管理内存块来满足不同大小的内存请求，并进行高效的合并和分割操作。其优点在于分配和释放内存块的操作非常快速，且通过内存块大小的选择和合并操作，有效减少了外部碎片。但是对于页面内部的空间碎片，需要更细粒度的管理器进行分配，考虑到空闲物理空间与内核堆空间应用场景的不同，我们选择使用不同的细粒度分配器进行管理。

2.3.2.2 内核物理内存

内核中的物理空间，即实际分配的物理页面是在操作系统中应用最多的空间，F7LY 使用对象化资源管理的思想，设计物理管理类 `PhysicalMemoryManager` 并创建全局对象 `k_pmm` 进行管理。

```
class PhysicalMemoryManager
{
public:
```

```

static void *alloc_page();
static void free_page(void *pa);
static void *kmalloc(size_t size);
static void *kcalloc(uint n, size_t size);
void clear_page(void *pa);
private:
    static BuddySystem *_buddy;
    static uint64 pa_start;
    // .....
};
extern PhysicalMemoryManager k_pmm;

```

`k_pmm` 在初始化时会使用地址布局中的地址标志 (`pa_start`)，原本 Buddy 的初始化放在这里，Buddy 变成 pmm 的一个成员 `pa_start` 是 buddy 系统在物理内存中的起始地址，加上一个 `Sizeof(BuddySystem)` 后后面存的东西是 tree，然后 tree 存完了之后才是 buddy 系统管理的那块内存。加上的 BSSIZE 是预留来放 BuddySystem 的大小和 tree 的大小，这之后才是 buddy 系统管理的那块内存，这时 `pa_start` 指向的就是 buddy 系统管理的那块内存的开始地址，再被初始化为 buddy 的基址。

```

void PhysicalMemoryManager::init()
{
    memlock.init("memlock");
    pa_start = reinterpret_cast<uint64_t>(end);
    pa_start = (pa_start + PGSIZE - 1) & ~(PGSIZE - 1);
    _buddy = reinterpret_cast<BuddySystem*>(pa_start);
    pa_start += BSSIZE * PGSIZE;
    memset(_buddy, 0, BSSIZE * PGSIZE);
    _buddy->Initialize(pa_start);
}

```

对于更细粒度的大小分配（函数 `kmalloc`），F7LY 采用了 linux 的 `SlabAllocator`。slab allocator 背后的思想是缓存经常使用的 object 并保持在初始状态供 kernel 使用。如果被基于 object 的 allocator，内核将耗费很多时间在分配，初始化和释放相同的 object。slab allocator 的目的就是缓存一些被释放的 object 因此这些基础的 structures 在多次调用期间被预留起来。

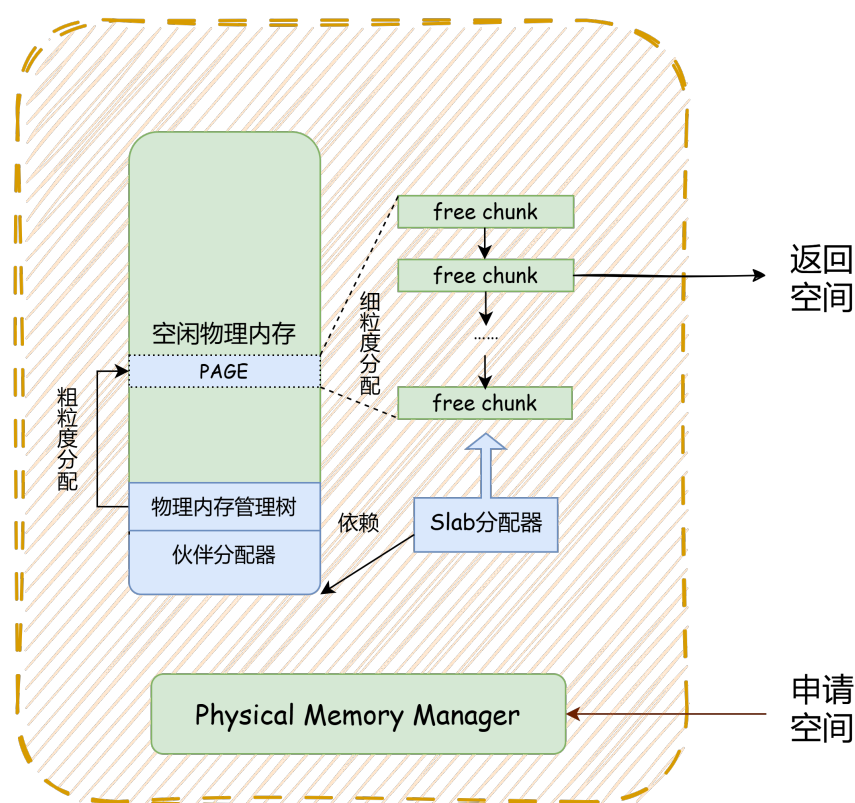


图 2-3 物理内存管理

slab allocator 由一组 cache 组成，这些 cache 由一个叫做 cache chain 的双向循环链表连接在一起。在 slab allocator 的上下文文件，一个 cache 就是一个管理许多像或者这种特殊类型的 object 的管理者。这些 cache 通过 cache struct 的 next 字段连接在一起。

每一 cache 维护了有多个连续物理 page 组成的 block，这些 block 称之为 slab。slab 被切分成很多小块来存放 slab 自身的数据结构和其管理的 object。

```
class SlabCache
{
private:
    static constexpr uint32
        ↪ DEFAULT_MAX_FREE_SLABS_ALLOWED = 5;
    uint32 obj_size_;
    uint32 free_slabs_count_;
    LinkedList<Slab> free_slabs_;
    LinkedList<Slab> partial_slabs_;
    LinkedList<Slab> full_slabs_;
}
```

```

}
.....

```

Slab 与 Buddy 二者分层次调用即可实现细粒度的物理内存分配，更小块的内存分配可以帮忙消除 buddy allocator 原本会造成的内部碎片问题。

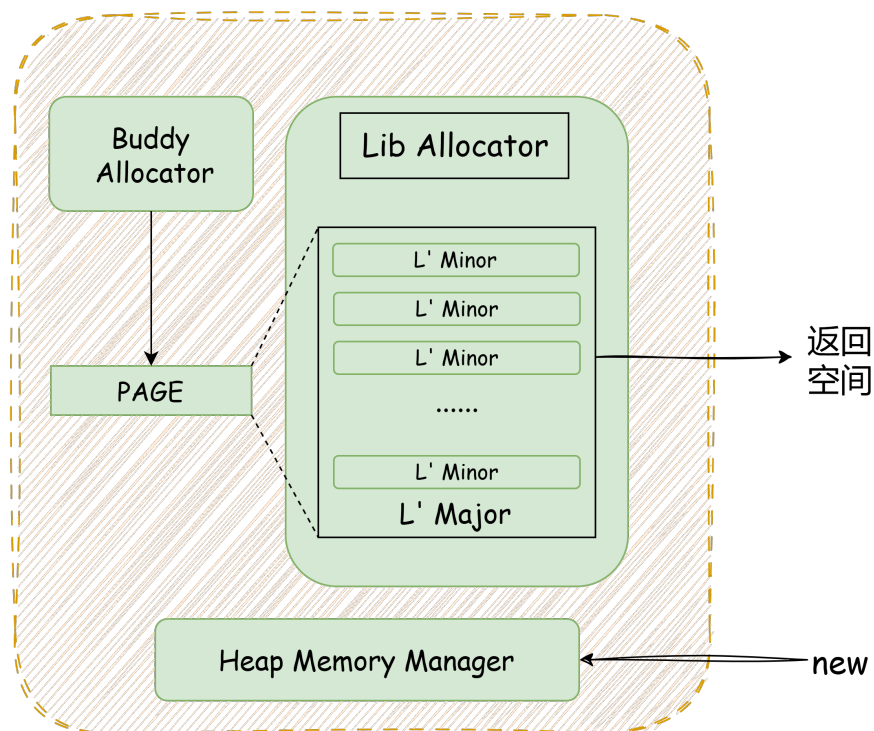


图 2-4 堆空间管理

2.3.2.3 内核堆内存分配

与物理内存类似，堆内存是 `cpp` 中用 `new` 动态创建对象的重要方法，为在内核环境中实现 `new` 与 `delete` 的正确功能，我们在内核中开辟一块堆空间，并重载 `new` 与 `delete` 来完成我们自己的对象实例化。

```

class HeapMemoryManager
{
private:
    SpinLock _lock;
    BuddySystem* _k_allocator_coarse;
    L_Allocator _k_allocator_fine;
public:

```



```

HeapMemoryManager() {};
void init(const char *lock_name, uint64_t
    ↪ heap_start);
void *allocate(uint64 size);
void free(void *p);
};
extern HeapMemoryManager k_hmm;

```

对于堆，设置管理类 `HeapMemoryManager` 并创建全局对象 `k_hmm` 进行管理。`k_hmm` 在初始化时会从堆地址开始初始化 Buddy System，并将此处划分为堆空间。

为提升堆分配的性能、减少碎片，我们在堆空间上引入了第三方高效内存分配器 `LibAllocator`^①，主要用于进行高效内存管理。分配器设计为二层结构：上层是**粗粒度分配器**（`L'Major`），负责大块内存获取；下层是**细粒度分配器**（`L'Minor`），用于将大块切分成适用于常规 `new / malloc` 调用的空间，极大提升了小对象分配的效率。

F7LY 对 `LibAllocator` 进行了定制性适配，使其不再从物理内存中直接申请页帧，而是通过内核内部的 `BuddyAllocator` 分配大块内存作为 `L'Major` 的来源。这种结构层次分明、责任明确，既保持了 `Buddy` 分配器的可控性，也引入了 `LibAllocator` 在细粒度分配上的高效策略。

F7LY 重载了标准的 `new / delete` 运算符，使其默认在堆空间上分配内存并交由 `k_hmm` 管理：

```

void * operator new(uint64 size)
{
    void *p = mem::k_hmm.allocate(size);
    return p;
}

void operator delete(void * p) noexcept
{
    mem::k_hmm.free(p);
}
// 省略展示其余用法的 new 和 delete 运算符。

```

① <https://github.com/blanham/liballoc>

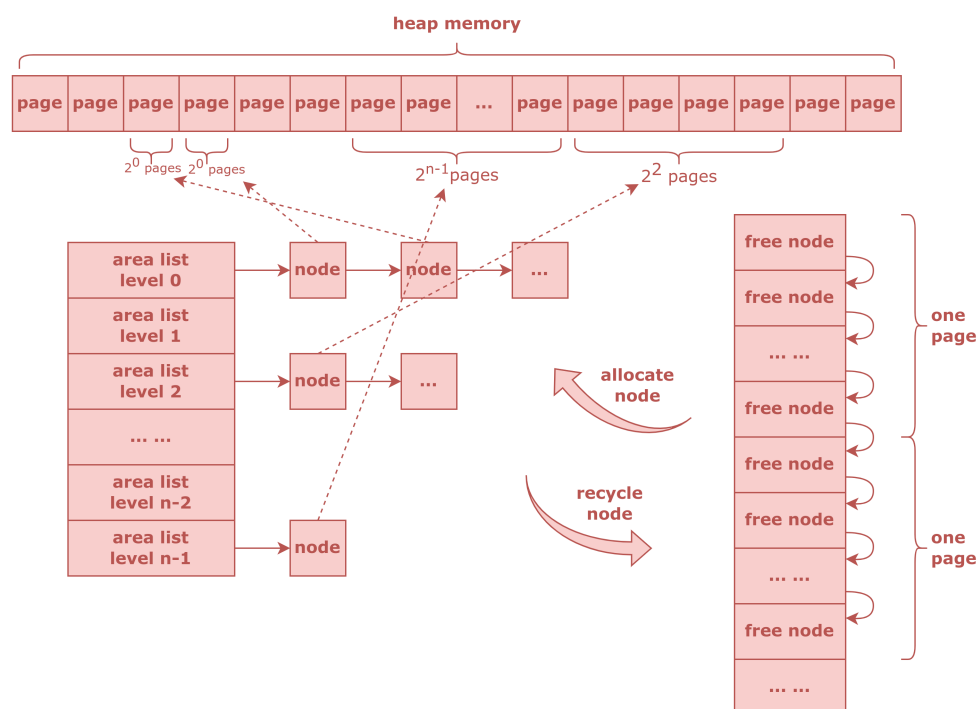


图 2-5 Buddy Allocator 分配示意图

通过这种方式，内核中的对象创建和销毁行为得以统一，所有堆内分配操作均通过 `k_hmm` 路由，有效避免了裸指针操作和分配器混用的问题。

2.3.2.4 地址空间管理

F7LY 采用**静态预配置 + 管理器分派**的机制进行地址空间管理。核心由 `VirtualMemoryManager` 管理器统一负责虚拟地址的分配、映射与回收，底层则由 `PageTable` 类协助完成页表操作与维护。这种结构设计不仅提升了内存管理的确定性和效率，也使得地址空间的生命周期管理更为清晰。

针对 `mmap` 所涉及的文件映射与匿名映射，F7LY 在每个进程控制块（PCB）中**预先分配了一块 VMA 表区域**，用于记录与管理该进程的虚拟内存区域（VMA），避免了运行时动态结构分配所带来的复杂性与不确定性。

页表 `PageTable` 类用于抽象和管理多级页表结构。其核心成员变量包括：

- `base_addr` 页表的物理基地址，指向页表的起始位置。
- `_is_global` 标记该页表是否为全局页表（如内核页表）。

主要接口包括：

- `walk(uint64 va, bool alloc)`：软件递归遍历多级页表，查找或分配虚拟地

址 `va` 对应的 PTE。可用于递归查找虚拟地址的映射关系，必要时分配中间页表。

- `walk_addr(uint64 va)` : 返回虚拟地址 `va` 映射的物理地址指针。
- `freewalk()` / `freewalk_mapped()` : 递归释放页表及其映射的物理页。
- `get_pte(index)` / `set_pte(index, pte)` : 获取/设置指定索引的 PTE。

在 `walk` 函数中，由于 RISC-V 使用 SV39 标准页表，而 loongarch 使用 4 级页表，二者不可统一，在此处 F7LY 分别定义了不同的实现，并在编译时根据宏进行区别。

PTE PTE 类封装了单个页表项 (Page Table Entry)，通常包含如下信息：

- 物理页帧号 (PPN)。
- 有效位 (Valid)、读/写/执行权限 (R/W/X)、用户/超级用户位 (U/S) 等标志位。

上述信息由于 loongarch 和 RISC-V 的架构不同而设置了不同的访问接口，可用于映射时设置或访问时查询。

虚拟内存管理器 VirtualMemoryManager 是内核虚拟内存的管理器，同样设置了全局对象 `k_vmm` 进行内存管理，它为进程和内核提供统一的虚拟地址空间分配、映射、回收、拷贝等功能，屏蔽了底层不同架构（如 RISC-V、LoongArch）页表实现的差异。

主要功能如下：

- **虚拟地址到物理地址的映射与解除映射**：通过 `map_pages`、`vmunmap`、`kvmmap` 等接口，将虚拟地址空间映射到物理内存，或解除映射关系。
- **内核页表的创建与初始化**：通过 `kvmmake` 创建内核页表，`kvmmap` 完成内核空间的初始映射。
- **用户空间内存分配与回收**：通过 `uvmmalloc`、`uvmdealloc` 等接口，支持进程虚拟空间的动态增长与收缩。
- **内存拷贝与字符串拷贝**：提供 `copy_in`、`copy_out`、`copy_str_in` 等接口，实现用户态与内核态之间的数据安全拷贝。

通过条件编译（如 `#ifdef RISCV`）自动适配不同架构下的页表实现，保证接口一致性与可移植性。

文件映射与匿名映射 针对 `mmap` 系统调用的匿名映射和文件映射，F7LY 采用在进程控制块中预留一片区域的方法进行管理。

```
struct vma
{
    int used;
    uint64 addr;
    int len;
    int prot;
    int flags;
    int vfd;
    fs::normal_file *vfile;
    int offset;
    uint64 max_len;
    bool is_expandable;
};
```

当用户态调用 `mmap` 系统调用时，内核会查找该进程 PCB 中的空闲 VMA 槽位，将映射信息记录其中，并返回分配好的虚拟地址。若为文件映射，则填入对应的 `vfile` 和 `offset` 字段；若为匿名映射，则标记 `flags` 中的 `MAP_ANONYMOUS`，并将 `vfile` 设为 `nullptr`。

2.3.2.5 缺页异常处理

F7LY 目前能够利用缺页异常处理来实现写时复制（Copy on write）、地址空间的懒分配（Lazy page allocation）以及用户的地址检查机制。

当用户程序因缺页异常进入内核时，两个架构的异常处理程序使用同样的处理逻辑，先检查缺页的地址是否处于物理空间，或处于 `vma` 记录的地址空间内，若是，则分配物理页面并建立映射。若不是，则抛出缺页错误。

2.4 进程管理

2.4.1 进程控制块 (PCB)

进程是操作系统中资源分配的基本单位。每个进程都有自己独立的地址空间和资源，如内存、文件描述符等。线程是操作系统中 CPU 调度的基本单位，线程共享所在的进程的地址空间和资源，但是有独立的执行上下文。

许多操作系统内核的设计都将进程和线程分开设计，分别使用 `Process` 和

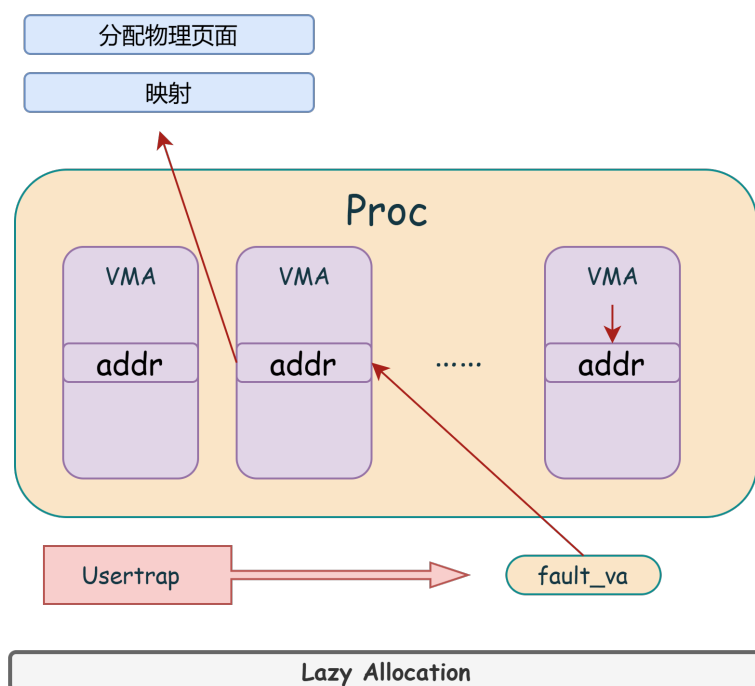


图 2-6 缺页异常处理流程

Thread 结构体来表示。但其实在 Linux 内核中并没有严格地区分进程和线程，而是通过一组统一的 API 来操作任务。例如 `sys_clone` 通过不同的 flags 组合创建共享不同资源的新任务，因此进程和线程的创建本质上是类似的。F7LY 此处统一使用 Pcb 结构体，可以通过标志位决定任务的具体属性，而不需要区分进程结构体和线程结构体。

2.4.1.1 PCB 的组成

```
class Pcb;
extern Pcb k_proc_pool[num_process];
```

下面介绍一下进程控制块结构中的各字段以及含义。

2.4.1.2 进程的状态

在所有的使用之中，进程需要的状态可简略分为 5 种，含义如下：

进程状态的切换如下：

- 创建进程池时构造函数中赋值 `UNUSED`
- `alloc_proc()` 从空闲进程中初始化时创建用户页表分配物理空间，并切换为

表 2-1 进程状态说明

状态	含义
RUNNING	正在运行或准备运行的任务。此状态下，任务占用 CPU，执行其代码
RUNNABLE	进程处于就绪队列中，当正在运行的进程放弃 CPU 或时间片耗尽后会按照队列获取 CPU
UNUSED	进程创建而来的状态，即初始状态
USED	进程分配时进行切换，在创建和就绪之间的状态
SLEEPING	时间片耗尽或等待某个事件。该状态下进程可以被信号唤醒
ZOMBIE	进程已终止，但 PCB 依然存在，以便父进程读取退出状态并等待回收

USED。

- `user_init()` 中初始化进程，并在所有初始化完成后切换为 RUNNABLE。
- 调度器从就绪队列中找到优先级最高的进程，开始运行，并将状态设为 RUNNING。
- 当被系统调用终止或时间片用尽时进程放弃 CPU，状态回到 RUNNABLE。
- 当 `futex` 或 `sleep` 系统调用让进程进入休眠时，进程切换为 SLEEPING，此时等待信号唤醒进程。
- 进程使用 `exit` 退出时状态切换为 ZOMBIE。

2.4.2 进程调度

为便于全局调用，F7LY 使用调度器 Scheduler 类以及全局对象 `k_scheduler` 进行进程调度管理，其中具体的调度方法使用了与 MIT xv6 类似的逻辑方法。

```
class Scheduler
{
private:
    SpinLock _sche_lock;
public:
    Scheduler() = default;
    void init(const char *name);
    void switch_to_proc(Pcb* p);    // 调度切换
    int get_highest_proirity();
    void start_schedule();
```

```

void yield(); // 进程放弃 CPU，状态从 RUNNING 改为 RUNNABLE
void call_sched();
};

```

2.4.2.1 上下文切换

汇编函数 `swtch.S` 用于管理一个到旧进程内核线程的用户-内核转换（系统调用或中断），一个到当前 CPU 调度程序线程的上下文切换，一个到新进程内核线程的上下文切换，以及一个返回到用户级进程的陷阱。

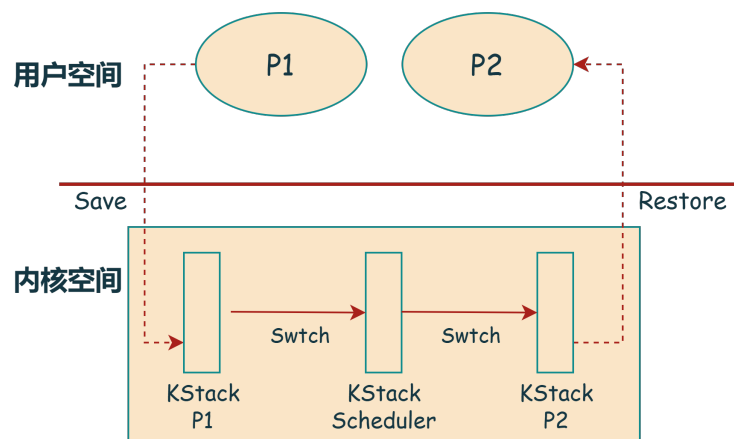


图 2-7 上下文切换

`swtch` 对线程没有直接的了解；它只是保存和恢复上下文（Contexts），`call_sched` 调用 `swtch` 切换到 `cpu->scheduler`，即每个 CPU 的调度程序上下文。调度程序上下文之前通过 `scheduler` 对 `swtch`（`swtch(&p->_context, cpu->get_context());`）的调用进行了保存。当我们追踪 `swtch` 到返回时，它返回到 `scheduler` 而不是 `sched`。

2.4.2.2 有栈协程调度

`Scheduler` 类承担了原本 c 语言书写的 xv6 中的调度任务，存在一种情况使得调度程序对 `swtch` 的调用没有以 `sched` 结束。一个新进程第一次被调度时，它从 `forkret` 开始。`Forkret` 存在以释放 `p->lock`；否则，新进程可以从 `usertrapret` 开始。

Scheduler（运行一个简单的循环：根据优先级找到要运行的进程，运行它直到它让步，然后重复循环。**scheduler** 在进程表上循环查找可运行的进程，比较它们的优先级并选出最高优先级，该进程具有 `p->state == RUNNABLE`。一旦找到一个进程，它将设置 CPU 当前进程变量 `c->proc`，将该进程标记为 `RUNNING`，然后调用 `swtch` 开始运行它。

这种用对象封装两个有栈协程 `call_sched` 和 `start_schedule` 的方法可以使 F7LY 比 xv6 有更清晰和高效的调度方法。

2.4.3 线程的支持

线程拥有独立的执行上下文，但处于同一个进程中，其核心特征是**共享地址空间**。在 F7LY 中，地址空间的映射由 `PageTable` 类统一管理。为了让线程共享地址空间，F7LY 采用最直观而高效的方式：在 `clone` 创建新 PCB（线程）时，直接使用所属进程的页表基址初始化新的 PCB 页表指针，使所有线程映射到同一地址空间，从而实现地址的完全共享。

然而，这种共享机制也引出了一个问题：

当单个线程退出时，若其直接释放自身的页表，将会导致其他线程因共用该页表而发生内存错误甚至崩溃。为了保证线程安全地释放地址空间，F7LY 为页表引入了**引用计数机制**：

```
class PageTable
{
private:
    uint64 _base_addr;
    int* _ref_count = nullptr;
    // .....
}
```

- 进程创建时，会将页表的引用计数初始化为 `1`，并为其单独分配一个 `int` 用于记录计数。
- 当通过 `clone` 创建线程时，新线程 PCB 会指向所属进程的页表，同时共用该计数指针，并将引用计数加一。
- 每当线程退出时，引用计数减一。仅当计数归零时，才由最后一个线程调用 `proc_freepagetable` 释放页表及其相关资源。

这一机制确保了即使多个线程并发退出，也能安全且正确地释放共享页表，

而不会发生悬挂引用或重复释放的问题。

2.4.4 进程操作与管理

F7LY 的进程管理使用类 `ProcessManager` 封装对进程的许多操作，并使用全局对象 `k_pmm` 进行管理，既包含了对于当前进程的状态信息获取和修改接口，又作为系统调用和进程结构体这一个个体之间的桥梁便于通过系统调用直接对进程进行操作。

下面将介绍这一类的公共接口与各字段的含义：

```
class ProcessManager
{
private:
    SpinLock _pid_lock;
    SpinLock _wait_lock;
    int _cur_pid;
    Pcb *_init_proc;
    uint _last_alloc_proc_gid;

public:
    ProcessManager() = default;
    // .....
}
```

执行用户态程序的流程是：首先使用 `fork` 创建一个子进程，然后通过 `execve` 加载 ELF 文件中的用户程序，将原进程的内存空间和执行上下文替换为新程序的资源，并从 ELF 文件定义的入口地址（`entry`）开始执行。

当用户态程序需要申请内核资源或执行特权操作时，会通过系统调用进入内核。这一过程由硬件触发用户态到内核态的陷入（`usertrap`），在陷入点内核会根据异常码进行判断，并通过 `syscall_handler` 包装逻辑进入具体的系统调用处理流程。

在 F7LY 中，系统调用的管理由专用的 `SyscallHandler` 类完成，并在内核启动的 `main` 函数中统一完成系统调用表的初始化绑定。其核心结构示例如下：

```
class SyscallHandler
{
    using SyscallFunc = uint64 (SyscallHandler::*)();
```

```
private:
    SyscallFunc _syscall_funcs[max_syscall_funcs_num];
    const char *_syscall_name[max_syscall_funcs_num];
    uint64_t _default_syscall_impl();
    // .....
}
extern SyscallHandler k_syscall_handler;
```

在系统调用流程中，SyscallHandler 的入口首先接收用户态传递的参数，然后分发到对应的系统调用处理函数。如果系统调用涉及进程管理相关操作（如 `fork`、`execve`、`wait` 等），SyscallHandler 会调用 `ProcessManager` 负责实际的逻辑执行，如创建/切换/终止进程。其余操作由于 SyscallHandler 已经进入内核态，可以在函数表中调用处理逻辑进行处理。

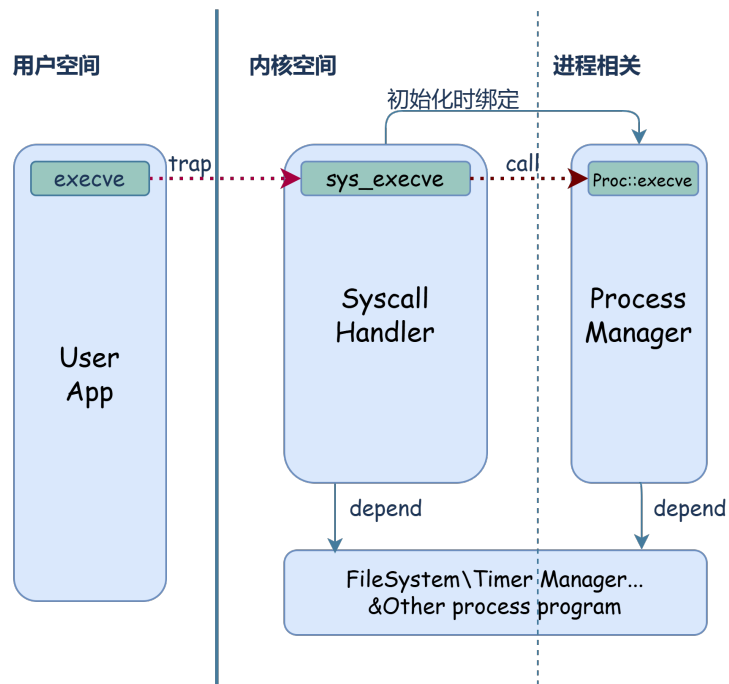


图 2-8 系统调用流程

这种分层结构增强了内核与用户态之间的边界控制，确保内核的安全性和可维护性以及 C++ 的封装和面向对象特性。同时，通过集中化的 `SyscallHandler` 类管理所有系统调用，也方便了系统调用表的维护和未来的功能扩展。

2.5 文件系统架构

2.5.1 虚拟文件系统

虚拟文件系统（Virtual File System, VFS）是操作系统内核中的关键子系统，负责统一管理磁盘文件、I/O 设备等字符流的交互，为上层提供一致、透明的文件访问接口。VFS 通过标准化的系统调用（如 `open()`、`read()`、`write()`）屏蔽了底层文件系统和存储介质的差异，使用户程序能够以统一的方式访问不同类型的文件系统。

F7LY 系统采用更符合现代编程范式的面向对象设计，使用 C++ 的多态特性构建 VFS 虚基类，并提供标准化的虚函数接口供各类文件系统提供具体的实现。多态机制使得虚拟文件系统层能够以完全透明的方式操作各种文件系统实例，无论是传统的 Ext4、NTFS 还是特殊的内存文件系统，对上层应用而言都呈现为完全一致的操作接口，这种抽象层次极大地简化了文件系统的管理和使用复杂度。

2.5.1.1 虚拟文件系统架构

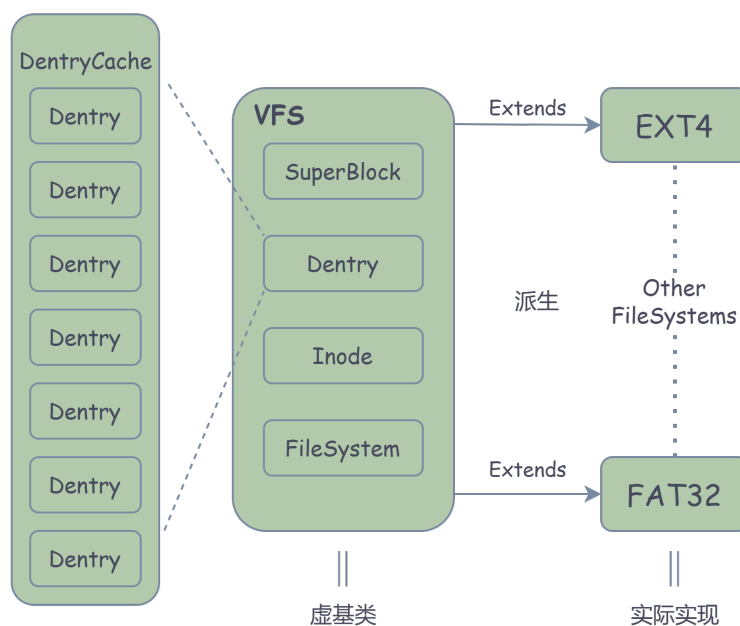


图 2-9 虚拟文件系统架构

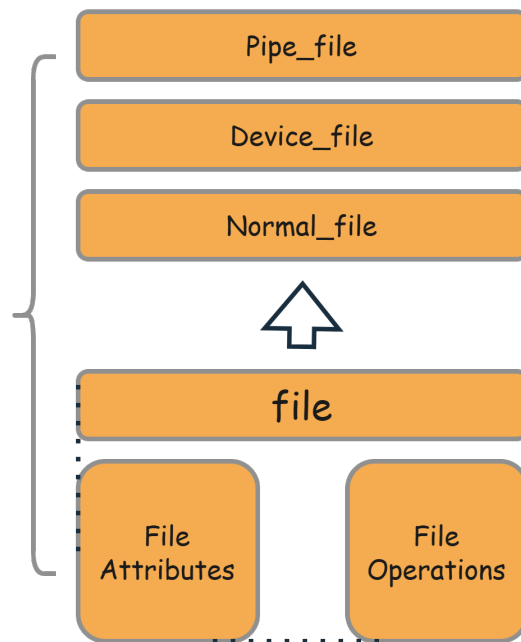


图 2-10 虚拟文件类型架构

2.5.1.2 虚拟文件类型架构

2.5.2 VFS 核心元数据结构剖析

2.5.2.1 Buffer

Buffer 用于管理磁盘数据的内存缓冲区。它提供了磁盘扇区数据在内存中的抽象表示。F7LY 的缓冲区容器（BufferBlock），组织和管理一组相关的磁盘缓冲区，用于组成了链表节点，串接而成一块缓冲数据链。

2.5.2.2 SuperBlock

超级块作为文件系统的核心元数据结构，承担着存储文件系统全局配置信息的重要职责。在物理存储层面，该结构通常映射到磁盘特定位置的元数据存储区域。从面向对象的角度来看，每个超级块实例都代表着一个具体的文件系统实例化对象。

针对基于持久化存储的文件系统，其生命周期管理包含以下关键流程：

- **挂载阶段：**内核需要从磁盘元数据区域读取原始超级块信息，并在内存中构建对应的运行时数据结构；
- **卸载阶段：**系统需要执行相反的操作，包括释放内存中的超级块对象，并将修改后的元数据同步回持久化存储设备。

而对于非持久化文件系统（如内存文件系统 sysfs、procfs 等），其超级块管

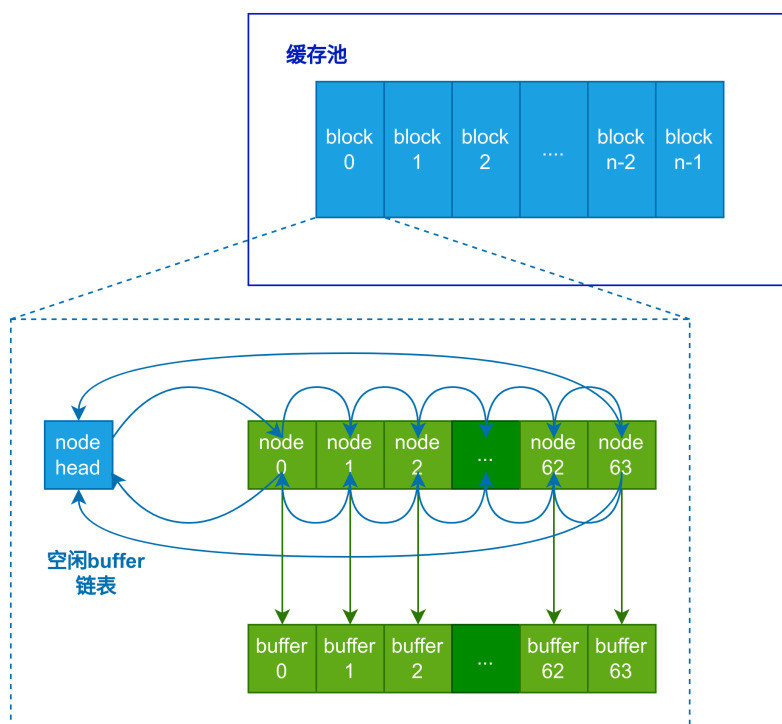


图 2-11 Buffer 缓冲区管理

理则简化为仅需在内存空间维护独立的元数据结构，无需考虑与物理存储设备的同步问题。这种差异化的实现机制充分体现了 VFS 设计对不同存储介质的良好适应性。

F7LY 的 VFS 超级块由 SuperBlock 定义并提供抽象接口：

```
class SuperBlock
{
public:
    SuperBlock() = default;
    SuperBlock(const SuperBlock& sb) = delete;
    virtual ~SuperBlock() = default;
    SuperBlock & operator=(const SuperBlock& sb);
    virtual dentry* getRoot() const = 0;
    virtual dentry* getMntPoint() const = 0;
    virtual FileSystem* getFileSystem() const = 0;
    virtual bool isValid() const = 0;
    virtual FileAttrs rDefaultMod() const = 0;
    virtual size_t rBlockSize() const = 0;
    virtual size_t rBlockNum() const = 0;
```

```
virtual Inode *allocInode(FileAttrs attrs,
    ↪ eastl::string dev_name = "") = 0;
};
```

对于具体的文件系统，只需要实现自己的超级块对象，完成继承对超级块基类的继承。以 Ext4 文件系统的派生为例。Ext4 文件系统通过 Ext4SB 类实现其特有的超级块管理机制，该类继承自 VFS 的 SuperBlock 基类，完整实现了 Ext4 文件系统的超级块管理功能。

```
class Ext4SB : public SuperBlock
{
    friend Ext4FS;
    friend Ext4BlockGroup;
private:
    Ext4SuperBlock _super_block;
    Ext4FS          *_belong_fs          = nullptr;
    bool            _support_64bit        = false;
    size_t          _cache_block_size     = 0;
    size_t          _cache_block_count    = 0;
    size_t          _cache_free_block_count = 0;
public:
    Ext4SB() = default;
    Ext4SB(Ext4SuperBlock *sb, FileSystem *fs);
    virtual ~Ext4SB();
    virtual dentry *getRoot() const override
        { return nullptr; };
    virtual dentry *getMntPoint() const override
        { return nullptr; };
    virtual bool isValid() const override
        { return _cache_block_size
            ↪ !=0; };
    // 其它的一些 override 的函数
    // .....
    size_t rBlockFree() const { return
        ↪ _cache_free_block_count; }
    bool support64bit() const { return _support_64bit; }
};
```

2.5.2.3 Inode

在 F7LY 中，Inode（索引节点）是文件系统的核心元数据结构，我们通过独立于文件名的 Inode 编号唯一标识每个文件或目录，并在其中存储文件类型、权限、所有者、大小、时间戳、数据块位置等关键信息。F7LY 的底层文件系统采用多级索引机制：直接指针用于快速访问小文件数据，间接指针支持大文件的存储扩展，从而兼顾性能与可扩展性。

在虚拟文件系统层（VFS），F7LY 实现了对 Inode 的抽象，使不同文件系统的 Inode 在保持各自特性的同时，对上层提供一致的操作接口。文件访问过程中，F7LY 的 VFS 会将磁盘上的 Inode 动态加载到内存中构建缓存对象，并管理其引用计数，以在保证访问性能的同时维持数据一致性。

F7LY 采用 Inode 与目录项（dentry）分离的设计，实现了对硬链接的支持：多个文件名可指向同一 Inode，只有当链接计数减至零时，系统才会真正释放文件占用的存储空间。实际文件操作中，无论是路径解析、权限检查，还是数据读写，F7LY 都依赖 Inode 提供的元数据支持，使 Inode 成为物理存储与逻辑文件访问之间的关键桥梁。

```
class Inode
{
public:
    SpinLock _lock; // inode lock, used for concurrent
        ↪ access control
    Inode() = default;
    Inode(const Inode& inode) = default;
    virtual ~Inode() = default;
    Inode& operator=(const Inode& inode);
    // 一些次要函数
    virtual size_t nodeRead(uint64 dst_, size_t off_,
        ↪ size_t len_) = 0;
    virtual size_t nodeWrite(uint64 src_, size_t off_,
        ↪ size_t len_) = 0;
    virtual size_t readSubDir(ubuf& dst, size_t off) =
        ↪ 0;
    virtual int readlinkat(char* buf, uint64 len) = 0;
    virtual FileAttrs rMode() const = 0;
    virtual dev_t rDev() const = 0;
    virtual uint64 rFileSize() const = 0;
    virtual uint64 rIno() const = 0;
```

```
virtual SuperBlock* getSb() const = 0;
virtual FileSystem* getFS() const = 0;
};
```

Ext4 等具体文件系统的派生与 SuperBlock 类似，很好的利用了现代编程语言的多态机制。

2.5.2.4 Dentry

Dentry（目录项）是管理文件路径信息的核心数据结构，它在内存中构建了一个完整的目录树结构，将文件系统的层次化路径抽象为可高效操作的树形节点。每个 Dentry 都代表文件路径中的一个组成部分，通过父子关系连接形成完整的路径树，其根节点通常是文件系统的挂载点。这种设计与 Unix“将文件名与文件本身分离”的哲学一脉相承——Dentry 存储路径名并指向对应的 Inode，而 Inode 则专注于管理文件实体数据，这种解耦设计使得硬链接（多个 Dentry 指向同一 Inode）成为可能。虽然某些简化设计（如 FAT32）会将路径解析功能直接整合到 Inode 中，但这种做法牺牲了系统的扩展性，无法支持现代文件系统的高级特性。在实际运行中，Dentry 与 Inode 形成多对一映射关系，系统通过维护 Dentry 缓存可间接缓存关联的 Inode，极大提升了路径查找效率。

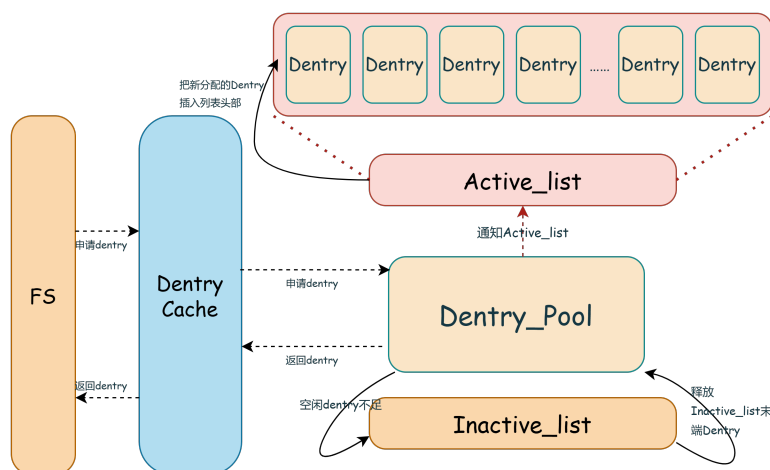


图 2-12 Dentry 缓存机制

在 F7LY 的文件系统中，DentryCache 负责缓存目录项（dentry），提高文件系统性能。通过缓存目录项，系统能减少磁盘 I/O，快速解析路径。


```

struct dentryCacheElement {
    fs::dentry dentry;
    bool is_active_;
    bool pin_;
};

class dentryCache {
    SpinLock _lock;
    list<dentryCacheElement*> leafList_;
    list<dentryCacheElement*> intenallList_;
    list<dentryCacheElement*> freeList_;
    dentryCacheElement* freeListHead_;
    dentryCacheElement
        ↪ dentryCacheElementPool_[MAX_DENTRY_NUM];
};

```

F7LY 的缓存机制通过维护空闲的队列和使用过的队列，缓存在 Cache 中使用过的 Dentry。

- freeList_: 存储空闲节点。
- leafList_: 存储叶子节点，用 LRU 策略管理。
- intenallList_: 存储固定的内部节点。

当缓存不足时，优先从 leafList_ 中回收未固定的节点从而起到加速缓存的效果。

同时，F7LY 设计并维护了一个专用的 `Path` 类，用于解析路径字符串并定位其对应的 dentry。通过构造 `Path` 对象，可基于当前工作目录或绝对路径，从根目录或指定起点开始查找路径对应的 dentry，方便用户空间与内核之间的路径解析和文件操作。

```

class Path
{
private:
    dentry *base;
    eastl::string pathname;
    eastl::vector<eastl::string> dirname;
public:
    Path(const eastl::string& path_, file *base_);
};

```

```
void pathbuild();
dentry *pathHitTable();
dentry *pathSearch(bool parent = false);
dentry *pathCreate(uint32 mode);
int mount(Path &dev, eastl::string fstype, uint64
    ↪ flags, uint64 data);
int umount(uint64 flags);
// .....
}
```

F7LY-OS 严格遵循 dentry-inode 分离设计，保持了与 Unix 文件系统的高度兼容性，并通过智能缓存策略在性能与一致性之间实现了良好平衡。这种路径解析、dentry 与 inode 协同工作的机制，不仅支持硬链接、快速路径查找等高级功能，也为多文件系统支持提供了强大的基础，构成了灵活高效的核心文件系统架构。

2.5.3 支持的文件系统

目前文件系统 F7LY 支持 Fat32 与 Ext4。

FAT32，全称为 File Allocation Table 32，是一种文件系统格式，用于在各种存储设备上存储和管理文件和目录。它是 FAT 文件系统的一个版本，最初由微软在 1996 年引入，主要是为了解决 FAT16 在处理大容量存储设备时的限制问题。FAT32 文件系统在 Windows 操作系统以及许多其他设备和媒体中得到了广泛应用。

FAT32 的主要特点包括：良好的稳定性和兼容性，大容量磁盘上的高空间利用率。分区容量限制可达 128TB、单个文件最大支持 4GB。

FAT32 文件系统的结构主要包括三个部分：

- **引导区**：包含文件系统的具体信息，如 FAT 表个数、每个 FAT 表的大小、每扇区内的字节数目等。
- **文件分配表区**：管理磁盘空间和文件，保存逻辑盘数据区各簇使用情况信息。
- **数据区**：存放用户数据，以簇为分配单位来使用。

Ext4，全称为 Fourth Extended Filesystem，是一种现代化文件系统格式，广泛应用于 Linux 操作系统中。Ext4 于 2008 年随 Linux 2.6.28 内核引入，是 Ext3 的继任者，旨在克服 Ext3 在大容量存储、性能和可靠性方面的局限性。如今，Ext4 已成为 Linux 发行版默认的文件系统，凭借其卓越的性能和稳定性，适用于服务

器、桌面以及嵌入式设备等多种环境。

Ext4 的主要特点包括：支持最大 1EB（Exabyte）级别的文件系统容量，单个文件大小可达 16TB；采用 Extents（区间）管理文件块，减少碎片并提升大文件访问性能；支持延迟分配（Delayed Allocation）等先进特性，有效优化磁盘写入；并具备日志功能（Journaling），在系统崩溃后能快速恢复文件系统一致性。

Ext4 文件系统的核心结构主要包含三个部分：

- **超级块（Superblock）**：保存文件系统整体的元信息，如块大小、总块数、inode 总数、挂载状态等关键参数。
- **块组描述符表（Block Group Descriptors）**：记录每个块组中空闲块、inode、位图区等信息，实现高效的空间管理和分配。
- **数据区**：包含 inode 表、位图、文件和目录的实际数据内容，Ext4 使用 Extents 技术将文件的连续数据块以区间形式存储，减少寻址开销并提高顺序读写性能。

F7LY 借鉴了 2024 武汉大学优秀开源作品的 FAT32 和 EXT4 实现，并使用了其改编自开源 c 语言 fatfs 与 ext4fs 库的文件操作。通过实现二者对应的 VFS 层接口完成了文件系统的对接。

2.6 进程间通信

2.6.1 信号机制

信号是操作系统向进程传递异步事件通知的重要手段，广泛用于异常处理、进程控制和进程间通信。F7LY 借鉴 Linux 的设计，支持符合 POSIX 标准的信号机制，并在 PCB 中内置信号处理所需的关键数据结构。

2.6.1.1 核心数据结构

F7LY 定义了以下结构体以支持灵活、兼容的信号处理：

```
namespace signal
{
    struct signal_frame
    {
        sigset_t mask;
        TrapFrame tf;
        signal_frame *next;
    };
    typedef struct sigaction
```

```

    {
        void (*sa_handler)(int); // 信号处理函数
        sigset_t sa_mask;         // 处理期间阻塞的信号
        int sa_flags;
    } sigaction;
}

```

这些结构体与 POSIX 标准中的 `siginfo_t` 语义接近，增强了对标准接口的兼容性；

- `signal_frame` 用于保存信号处理过程中的进程上下文；
- `sigaction` 定义每个信号对应的处理行为。

F7LY 的信号机制支持包括 `SIGCHLD` 在内的常用信号。例如，当子进程状态变为 Zombie 时，内核通过发送 `SIGCHLD` 信号告知父进程状态变化，同时提供子进程的 PID、状态码、用户态和内核态运行时间等信息，为父进程调用 `wait4()` 等系统调用获取子进程状态提供必要数据。

2.6.2 信号处理函数

在任务从内核态返回用户态前，需要处理挂起的信号。F7LY 在中断处理器 `usertrap` 中加入 `handle_signal()`，在时钟中断或其他中断后及时执行挂起信号：

```

if (which_dev == 2)
{
    timeslice++;
    if (timeslice >= 10)
    {
        timeslice = 0;
        printf("yield in usertrap\n");
        proc::k_scheduler.yield();
    }
}
handle_signal();
usertrapret();

```

`handle_signal()` 的核心逻辑是：

1. 获取当前进程 PCB;
2. 遍历进程挂起的信号位掩码, 检查每个信号的有效性;
3. 根据 `sigaction` 判断信号是否需要自定义处理:
 - 若 `sa_handler == nullptr`, 使用默认处理 (例如 SIGKILL 设置 `p->_killed=true`);
 - 否则调用用户自定义的信号处理函数;
4. 处理完当前信号后不会丢失其他信号, 通过位掩码保留剩余信号, 保证处理安全性。

2.6.2.1 信号上下文保存与恢复

F7LY 实现的信号处理支持完整的上下文保存与恢复:

- 当内核准备进入用户态执行信号处理函数时, 会将当前上下文保存到 `sig_trapframe` 中;
- 信号处理完成后, 程序需回到原执行状态。此过程使用 `sigreturn` 系统调用恢复上下文。

为了实现从信号处理返回, F7LY 设计了类似 `trampoline` 的机制:

- 信号处理时将 EPC (返回地址) 设置到汇编实现的 `sig_trampoline` 函数;
- `sig_trampoline` 唯一作用是触发 `ecall`, 调用 `SYS_rt_sigreturn` 系统调用, 执行上下文恢复。

这一设计使得 F7LY 在支持自定义信号处理时能保证上下文正确性, 安全地完成用户态与内核态的切换。

2.6.3 Futex

Futex (Fast Userspace Mutex) 是 Linux 内核提供了一种高效用户态同步原语, 可用于实现互斥锁、条件变量、信号量等多种同步机制。F7LY 借鉴 Linux 的设计, 构建了可扩展的 Futex 实现, 并支持相关系统调用。

```
struct robust_list {
    struct robust_list *next;
};
struct robust_list_head {
    robust_list list;           // 链表头
    long futex_offset;         // futex 字段的相对偏移
```

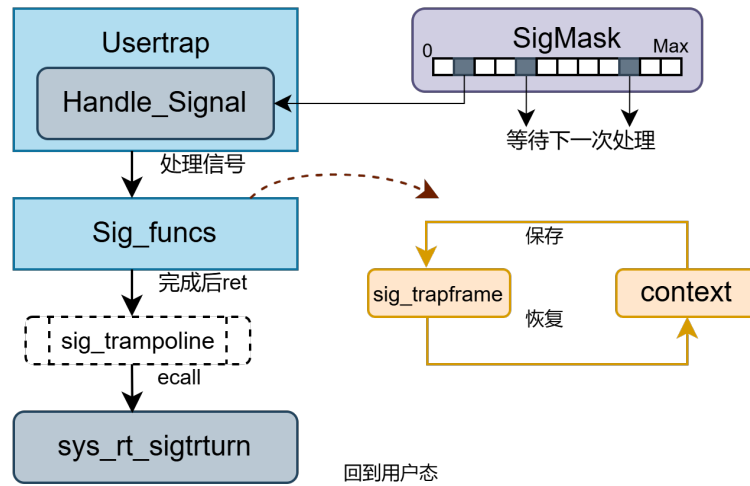


图 2-13 信号处理流程

```
robust_list *list_op_pending; // 待处理的锁操作
};
```

F7LY 使用 `robust_list` 结构体作为构成单向链表的节点嵌入在用户空间的锁结构中。内核只需要知道前向链接，用户空间可以使用双向链表实现 $O(1)$ 的添加和删除。

- `list` : robust locks 链表的头节点，如果为空则指向自己。
- `futex_offset` : 用户空间设置的相对偏移量，告诉内核 `futex` 字段在数据结构中的位置。
- `list_op_pending` : 防止线程死亡竞争的字段。
 - 用户空间首先将此字段设置为即将获取的锁的地址。
 - 然后执行锁获取操作。
 - 再将自己添加到列表中。
 - 最后清除此字段。

Robust Futex 的意义是当线程异常终止时，内核可以自动清理该线程持有的锁，防止死锁。

```
int futex_wait(uint64 uaddr, int val, tmm::timespec
  ↪ *timeout);
```

```
int futex_wakeup(uint64 uaddr, int val, void *uaddr2,  
↳ int val2);
```

Futex 是现代多线程程序同步的基础设施，可用于实现 pthread 库的互斥锁、条件变量、读写锁等高级同步原语，既具备高效的用户态自旋性能，又能在需要阻塞时与内核协作实现调度，让同步操作兼具性能和安全性。

第三章 总结与展望

3.1 工作总结

1. 完成 RISC-V 与 LoongArch 双架构支持，包括：
 - 串口驱动；
 - PCI 设备管理；
 - 中断控制器配置；
 - 硬盘读写；
 - 并实现与硬件逻辑相匹配的中断处理流程。
2. 通过面向对象设计，完成进程管理和内存管理模块，实现底层架构无关性，达到松耦合结构，为双架构扩展奠定基础，并支持线程任务的创建与调度。
3. 实现独立的虚拟文件系统层，成功将文件系统逻辑与内核解耦，并为支持更多文件系统格式提供继承接口。
4. 完成 Busybox 和 libc 的适配，补充和完善系统调用接口与相关功能。
5. 通过初步性能测试，验证系统具备良好的调度和 I/O 性能表现。
6. 基础实现进程间通信（IPC）机制和信号处理功能，支持简单的进程协作。
7. 构建多等级调试日志系统，并支持内核崩溃时打印调用栈回溯（backtrace）及参数信息，方便问题定位。

3.2 经验总结

1. 积累从零开始构建内核的系统化经验，包括：
 - 完成机器引导；
 - 初始化串口输出；
 - 搭建内存布局；
 - 编写中断处理；
 - 实现最小可用内核；
 - 逐步完善核心功能。

2. 系统调用的实现结合《System Calls Manual》和参考其他优秀项目成果，符合 Linux 原生 raw syscall 规范。
3. 不断积累调试经验，包括：
 - 用户态程序的单步调试；
 - 利用 GDB 跟踪调用栈；
 - 精准定位出错变量；
 - 依靠完善的日志获取运行期关键信息。

3.3 未来计划

1. 优化内存布局，进一步提升内存空间利用率。
2. 根据决赛可能提供的硬件平台，适配并完善必要的驱动程序。
3. 针对 Linux Test Project (LTP) 压力测试结果，改进内核结构以提升稳定性和兼容性。
4. 实现完善的网络协议栈，通过决赛测例验证网络功能。

附录 A 一些重要的类的具体字段解析

表 a-1 ProcessManager 接口函数说明（第一部分）

函数名	含义
get_cur_pcb	获取当前执行中的进程控制块
alloc_pid	为进程分配进程 ID
alloc_proc	从进程池中寻找一个空进程并分配
set_killed	将 killed 字段设为 1，等待杀死进程
proc_pagetable	为进程分配一个页表的空间并进行映射
proc_freepagetable	释放一个进程的页表并回收空间
either_copy_in	从用户空间或内核空间拷贝数据到内核空间
either_copy_out	从内核空间或用户空间拷贝数据到用户空间
进程操作	
execve	执行新程序
growproc	扩展进程内存，由 brk 等函数调用
wait4	等待子进程结束
load_seg	加载程序段到页表映射的虚拟内存中，在执行程序时使用
sleep	使进程进入睡眠状态，等待某个通道或锁
wakeup	唤醒在指定通道或锁上睡眠的进程

表 a-2 ProcessManager 接口函数说明（第二部分）

exit_proc	真正退出进程的逻辑，设置状态并清理资源
exit	当前进程或线程退出（只退出自己）
exit_group	当前线程组（或进程组）全部退出
clone	创建新进程或线程，支持克隆和线程局部存储
fork	创建新进程，返回新进程的 PID
brk	扩展进程地址，设置进程的堆顶地址
文件操作	
open	打开文件或设备，返回文件描述符
mkdir	在指定路径下创建目录
close	关闭文件描述符
fstat	获取指定文件描述符下的文件状态信息
chdir	改变当前工作目录
getcwd	获取当前工作目录路径
mmap	映射文件到内存
munmap	解除内存映射
unlink	删除文件或目录的链接
pipe	创建管道，返回读写文件描述符
其他系统调用	
reparent	托孤机制，重新设置进程的父进程为 init
prlimit64	设置或获取进程资源限制
set_robust_list	设置鲁棒列表，用于处理线程安全的锁
set_tid_address	设置线程 ID 地址，用于线程同步和管理

表 a-3 PCB 结构字段说明（第一部分）

字段名	类型	含义
<code>_lock</code>	SpinLock	进程控制块的锁，用于并发访问控制
<code>_gid</code>	int	全局 ID，用于在进程池中唯一标识进程
文件系统相关		
<code>_cwd</code>	dentry*	当前工作目录
<code>_cwd_name</code>	string	工作目录名称
<code>_ofile</code>	file*	进程打开的文件列表 (文件描述符 文件结构)
<code>_fl_cloexec</code>	bool	记录每个文件描述符的 close-on-exec 标志
<code>exe</code>	string	可执行文件的绝对路径
进程状态信息		
<code>_state</code>	enum Procstate	进程当前状态
<code>_chan</code>	void*	进程睡眠时等待的通道 (例如：某个锁或事件)
<code>_killed</code>	int	进程是否被标记为 kill (非零表示被 kill)
<code>_xstate</code>	int	进程退出状态，用于父进程 wait() 获取
<code>_pid</code>	int	进程 ID (Process ID)
<code>_parent</code>	Pcb*	父进程的 PCB 指针
<code>_name</code>	char	进程名称 (用于调试)

表 a-4 PCB 结构字段说明（第二部分）

字段名	类型	含义
内存管理相关		
<code>_kstack</code>	uint64	内核栈的虚拟地址
<code>_sz</code>	uint64	进程用户空间的内存大小 (字节)
<code>_pt</code>	PageTable	用户空间的页表
<code>_trapframe</code>	TrapFrame	保存用户态 TrapFrame 的地址 (用于系统调用和异常处理)
<code>_context</code>	Context	保存进程的上下文信息 (寄存器等), 用于进程切换
调度相关		
<code>_slot</code>	int	分配给进程的时间片剩余量
<code>_priority</code>	int	进程优先级 (0 最高, 19 最低)
消息队列		
<code>_mqmask</code>	uint	用于标记进程使用的消息队列
虚拟内存区域 (VMA)		
<code>_vm[NVMA]</code>	vma	虚拟内存区域数组
线程/futex 相关		
<code>_futex_addr</code>	void *	用于 futex 的唤醒
<code>_set_child_tid</code>	int*	线程的标志 id
<code>_clear_child_tid</code>	int *	清除线程表示 id 的 flag
<code>_robust_list</code>	robust_list_head *	用于处理线程的 robust futexes
信号处理相关		
<code>_sigactions[]</code>	sigaction *	信号处理函数数组
<code>sigmask</code>	uint64	信号掩码
<code>_signal</code>	int	进程等待的信号